TECHNICAL REFERENCE V2.2

Shokunin
The AI Engineering Ecosystem

Multi-strategy recall, explicit session management, 38 skills, and declarative ecosystem management.

swagger
Version 4.2.1 · 2026.05
github.com/EliasOulkadi/shokunin

Contents

01	Executive Summary	03
02	Architecture	04
03	Memory System	06
04	Hindsight Comparison	08
05	Skills Ecosystem	10
06	Update System	11
07	Quick Start	13

Executive Summary

Shokunin is a fully local AI engineering ecosystem built on OpenCode. It gives your AI agent persistent memory across sessions, 38 specialized skills, and a self-maintaining architecture. Version 4.2.1 adds multi-strategy memory recall (vector search alongside keyword matching, temporal filtering, and result fusion).

Two design choices set it apart. First, memory operations run locally: no API calls per store, no PostgreSQL, no Docker containers. A single shell script installs everything. Second, the ecosystem maintains itself through a declarative manifest and drift detection system. When components drift from their expected state, the update system catches it before you do.

We looked hard at Hindsight (vectorize-io/hindsight) for inspiration. Its multi-strategy retrieval is state of the art. We took the parts that make sense for a local tool (BM25 keyword search, temporal filtering, reciprocal rank fusion) and left the parts that would add server dependencies (LLM per memory operation, PostgreSQL, Helm charts). Section 04 has the full comparison.

Key Metrics

Metric	Value
Core scripts	17 (13 PowerShell + 4 Bash)
Managed components	24 (manifested, hash-tracked)
Skills	38 (across 8 domains)
MCP tools	5 (store, search, multi_search, consolidate, session_summary)
LLM providers	OpenCode Go + Ollama (local fallback)
Memory entries	36 (growing, ~5 per session)
Healthcheck	25/25 checks passing
Install size	Under 200 MB (including ChromaDB and embedding model)

02 · ARCHITECTURE

Architecture

Seven layers. Each with clear boundaries. No layer depends on the layer above it.

Layer Stack

Layer	Path	What It Does
Config	`~/.config/opencode/`	Provider setup, MCP server config, agent definitions, skill discovery paths
Instructions	`~/AGENTS.md` + `~/.claude/CLAUDE.md`	Global behavior rules, coding conventions, memory system instructions
Update	`~/.shokunin/`	Declarative manifest, drift detection, safe updates with automatic backups
Orchestration	`~/.shokunin/scripts/`	Session wrapping, memory persistence, healthchecks, cleanup automation
Memory	`~/.shokunin/memory/`	ChromaDB vector store, MCP server, session files (dual-interface: CLI + JSON-RPC)
Skills	`~/.config/opencode/skills/` + `~/.agents/skills/`	38 SKILL.md files with scripts, references, and assets
Infrastructure	`~/.shokunin/backups/` + `~/.shokunin/logs/`	Backups, operational logs, ChromaDB database files

Request Flow

A session starts with `run-opencode.ps1` (or `run-opencode.sh` on Linux). This wrapper sets session ID, environment variables, and a checkpoint timer. OpenCode loads its config including MCP servers. Skills are discovered from the skills directories. As the session progresses, checkpoints save to ChromaDB every 5 minutes. On exit, the wrapper captures the console buffer, parses it, and saves a session summary.

Memory Interfaces

The same ChromaDB database is accessible through two interfaces. `chroma-helper.py` runs as a CLI script for fast batch operations. `mcp-server.py` exposes the same data through JSON-RPC, which the AI agent calls via the `memory` MCP server. Both write to the same markdown fallback files, so if ChromaDB is unavailable, session data persists.

Interface	Protocol	Best For
`chroma-helper.py`	CLI (stdin/stdout)	Scripting, batch operations, healthchecks
`mcp-server.py`	JSON-RPC (stdin/stdout)	AI agent tool calls via MCP protocol

> The MCP server starts and stops per request (stdin/stdout). It never runs as a persistent daemon. This keeps resource usage minimal when not actively in a session.

Memory System

Version 4.2.1 introduces multi-strategy recall: vector similarity, BM25 keyword search, temporal filtering, and reciprocal rank fusion (combined from Hindsight's approach but adapted for a zero-server runtime).

How Recall Works Now

Before 4.2.1, memory search was a single ChromaDB similarity query. That missed exact keyword matches, had no time filtering, and occasionally returned noisy results. The new `recall` command runs multiple strategies and fuses them:

1. Vector search (ChromaDB query, same as before)
2. BM25 keyword search (inverted index on session markdown files, pure Python, zero dependencies)
3. Temporal filter (ISO date range, applied to both result sets)
4. Reciprocal Rank Fusion (RRF at k=60, merges and reorders the combined results)

This catches terms that vector search would miss. If someone stores "we discussed the Q3 budget" and you search "Q3 budget", BM25 finds it immediately because the exact words match. Vector search might return a semantically similar entry about "quarterly planning" instead.

Using Recall

```
# Basic recall (vector + BM25, fused)
python ~/.shokunin/scripts/chroma-helper.py recall "Q3 budget" "shokunin" 5

# With date filter
python ~/.shokunin/scripts/chroma-helper.py recall "memory" "" 10 2026-05-14 2026-05-15
```

The same capability is available through the MCP tool `multi_search_context`, which the AI agent can call directly:

```
# From within an AI session
search_context(query="API key", n_results=5, from_date="2026-05-10")
```

Memory Consolidation

Old entries get noisy. The `consolidate` command groups entries by project and extracts key terms by frequency. No LLM needed (it does TF-based keyword extraction). Run it periodically or let the agent trigger it automatically.

```
python ~/.shokunin/scripts/chroma-helper.py consolidate "shokunin"
# Output: {"consolidated": 3, ...}
```

MCP Tools

Tool	Description	Since
`store_context`	Save text with type, tags, project, session_id	v4.0
`search_context`	Vector similarity search with project/tag filters	v4.0
`multi_search_context`	Vector + BM25 + temporal filter + RRF fusion	v4.2.1
`get_session_summary`	Summarize all entries in a given session	v4.0
`consolidate_memories`	Group old entries by project with keyword extraction	v4.2.1
`list_sessions`	List recent sessions with project, entry count, and summary	v4.2.2
`continue_session`	Load full context from a specific session to continue	v4.2.2
`save_message`	Record individual message exchanges in session transcript	v4.2.2

> The memory system has a three-tier fallback: ChromaDB vector database (primary), markdown files in `memory/sessions/` (backup), and raw console buffer logs (emergency). If any tier fails, the next one takes over silently.

Session Management

Version 4.2.2 adds explicit session control. Instead of guessing which session to resume via text search, the agent lists recent sessions at startup and lets you choose.

```
# List sessions (date, project, entry count, summary)
python ~/.shokunin/scripts/chroma-helper.py session list 5

# Continue a specific session (loads all entries)
python ~/.shokunin/scripts/chroma-helper.py session continue session-20260515-1234

# Save a message to session transcript (for full capture)
python ~/.shokunin/scripts/chroma-helper.py session save "message" session-xxx "user"
```

The `session list` command returns structured metadata (session ID, timestamp, project, entry count, types used, summary snippet). The user picks a session by number, and `session continue` loads the complete context: session end summaries, decisions made, files modified, checkpoints. Messages are stored in JSONL format at `memory/sessions/<id>.jsonl`.

> This eliminates the biggest pain point in agent memory: trying to find the right context with a text search when you don't know what to search for. Now you see a list and pick.

Hindsight Comparison

We evaluated Hindsight (vectorize-io/hindsight) after community feedback. It is the most accurate agent memory system on benchmarks (91.4% on LongMemEval with its largest model). Some of its ideas directly improved Shokunin. Others we chose not to adopt, for reasons grounded in our design goals.

What We Learned From Hindsight

Hindsight Feature	What We Did	Why
Multi-strategy recall (4 parallel retrievals)	Added BM25 + temporal filter to existing vector search, fused with RRF	Catches exact keyword matches and time-bound queries that pure vector search misses
Reciprocal Rank Fusion	Implemented RRF at k=60 for merging vector and BM25 results	Simple algorithm, no extra dependencies, works better than raw score averaging
Cross-encoder reranking	Not implemented yet (low priority for current scale)	Reranking helps with large result sets; our current hit count doesn't justify the extra model
Memory consolidation / reflection	Added `consolidate` command for keyword extraction	Simple TF-based summarization works for local scale; full reflection requires LLM calls

What We Chose NOT to Adopt

Hindsight Feature	Why We Passed
LLM per `retain()` call for entity extraction	Adds cost, latency, and external API dependency. Shokunin stores raw text immediately without any LLM step. Tags provide cheap structure without calling a model.
PostgreSQL + pgvector	Server process requirement. ChromaDB gives us file-based vector storage at zero configuration. No database user, no connection strings, no Docker.
Graph retrieval (entity links, relationship traversal)	Requires maintaining an entity graph. Human-scale session memory (dozens of entries per session) doesn't benefit from graph traversal the way enterprise agent workloads do.
Docker / Helm / Kubernetes deployment	Shokunin installs with a single shell script. Adding container orchestration would be the opposite of our design goal.

Side-by-Side

Aspect	Shokunin v4.2.1	Hindsight v0.6.2
Vector storage	ChromaDB (file-based, 30 MB)	PostgreSQL + pgvector (server)
Keyword search	BM25 (in-memory, zero deps)	BM25 (PostgreSQL)
Retrieval strategies	2 (vector + BM25) + temporal	4 (semantic, BM25, graph, temporal)
Result fusion	Reciprocal Rank Fusion	RRF + cross-encoder reranking
Entity extraction	Manual tags	LLM-based, automatic
LLM dependency	None for memory operations	Required per retain
Install complexity	1 shell script	Docker run or pip + PostgreSQL
Offline capability	Full (no network needed)	Partial (needs LLM API)
Benchmarks	Not evaluated	91.4% LongMemEval

> Hindsight is better for enterprise agents that need state-of-the-art memory accuracy at scale. Shokunin is better for individual developers who want persistent AI memory without servers, API costs, or setup friction. They solve overlapping problems at different scales.

Skills Ecosystem

38 skills across 8 domains. Each is a self-contained `SKILL.md` file with optional scripts, references, and assets.

By Domain

Domain	Count	Skills
Infrastructure	5	docker, kubernetes, terraform, ci-cd, db-admin
Backend	4	api-forge, auth-architect, db-sculptor, error-handler
Frontend	6	component-forge, responsive-engine, motion-craft, landing-craft, ui-ux-pro-max, aesthetic-web
Mobile	2	flutter, react-native
Quality	3	test-commander, performance-profiler, code-review
Content	6	communication, content-marketing, business-proposals, seo-geo, translate-craft, documentation
Productivity	8	git-workflow, windows-powershell, runbook-gen, strategy, design, finance, legal-counsel, whendone-plus
Documents	4	kami, portfolio-auto, shokunin-update, chromadb

Skill Discovery

OpenCode discovers skills from `~/.config/opencode/skills/`, `~/.agents/skills/`, and `~/.claude/skills/`. Each skill is validated through a CI pipeline that checks for required sections (YAML frontmatter, workflow, error handling, sources).

Update System

A declarative approach to ecosystem maintenance. One file describes everything. Three commands manage it.

The Manifest

`~/.shokunin/shokunin.json` defines 24 components across 5 groups: scripts, linux_scripts, memory, configs, and protected paths. Each component tracks its path, type (static/template/data), runtime, and template reference. Variables like `{{SHOKUNIN_DIR}}` avoid hardcoded paths.

Commands

```
# Check drift
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" status

# Preview what would change (no modifications)
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" plan

# Apply with automatic backup
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" apply -Confirm

# Rollback to a previous state
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" rollback -Timestamp 20260514-
120000
```

Protection Rules

Four paths are marked `NEVER_MODIFY`: chroma_db, sessions, logs, and backups. The engine checks this rule before every operation. No command (not even apply) can touch them.

> Every `apply` creates a timestamped backup before modifying any file. Rollback restores from the backup. The update system maintains itself: it is included in the manifest as a managed component.

Quick Start

Install (Linux)

```
bash <(curl -sL
https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)
```

Flags: `-y` for non-interactive mode. NVIDIA key is optional (OpenCode Go works without it).

Install (Windows)

```
# Clone and run
git clone https://github.com/EliasOulkadi/shokunin.git
cd shokunin
powershell -File install.ps1
```

Verify

```
# Linux
~/.shokunin/scripts/linux/memory-healthcheck.sh

# Windows
& "$env:USERPROFILE\.shokunin\scripts\memory-healthcheck.ps1"

# Both: expect 25/25 checks passing
```

Daily Use

```
# Start a session
opencode

# Search memory from command line
python ~/.shokunin/scripts/chroma-helper.py recall "topic"

# Check ecosystem health weekly
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" status
```

> Full documentation and source at github.com/EliasOulkadi/shokunin. Community feedback welcome.